Learning through tinkering: Designing interactive worked examples for introductory-level data science students

Anna Fergusson
University of Auckland
New Zealand
a.fergusson@auckland.ac.nz

Sven Hūsing
Paderborn University
Germany
sven.huesing@uni-paderborn.de

Keywords: Data science education, Worked examples, Code blanks, Epistemic programming

1 Introduction

There is growing recognition of the need for effective and scalable approaches for teaching computer programming to data science students. While there are many packages, tools, and platforms that data science instructors can use to design and deploy interactive coding tasks, research literature within data science education that specifically addresses the pedagogical design of these tasks remains limited. In this paper, we draw on research literature from statistics and computing education, and examples from our own teaching, to explain our approach to designing interactive worked examples for introductory-level data science students.

2 Background

A common thread to discussions about data science education is that students need to integrate both statistical and computational thinking to learn from data, which necessitates students developing at least some computer programming skills. For initial experiences with computer programming, it is important to embed the use of code within a task that is purposeful and engages data science students. In this context, epistemic programming denotes a programming practice being taught not primarily as a professional practice, required for future computer science or data science careers, but as a skill that empowers students to engage in personally or socially meaningful projects. A common application in data science education is the exploration of data, conducted through an intertwined process of knowledge acquisition and programming. In this "tinkering" approach, students modify code, observe the resulting behaviour, and then make further code adjustments based on their reflections (Hüsing et al., 2023). Because this practice addresses programming novices, there is a need for scaffolds that introduce students to relevant domain-specific programming constructs. Worked examples are a common instructional resource in this regard. They offer context and guidance for data exploration and statistical modelling, and provide a code base that students can use, adapt, and extend for other purposes (Atkinson et al., 2000).

Interactive worked examples. With the increased availability and capabilities of computational tools and cloud-based platforms such as WebAssembly and Jupyter notebooks, instructors are able to incorporate interactivity within these worked examples. The most common case is providing editable code within the worked example that can be executed directly within the web browser (Fergusson, 2023). Other interactive aspects include providing auto-marked quiz questions, or code exercises that can be auto-graded against an accepted solution.

Using faded approaches. Another common approach is the "fading out" of the worked examples. Here, the provided code aspects of the worked examples are progressively faded (removed and replaced with blanks) so that the students themselves have to complete them. Decisions about which aspects should be faded in a worked example should be based on an analysis of the internal structure of the learning content (Renkl et al., 2004).

Integrating purposeful scaffolding. When designing worked examples, instructors need to consider the purpose of the code and which parts of the code should be changed to accomplish a purpose. Programming plans provide the code needed to produce a required output for a purpose, with scaffolding that directs learners' attention to the specific parts of the code that could be adapted or extended (Cunningham et al., 2021). The programming plans should also be supported by explanations so that students are aware for which goals they can be used.

Linking to projects. To motivate student engagement with worked examples, they should be followed up by students working on their own project (Sweller and Cooper, 1985), which is similar in structure, but requires adapting the code so there are opportunities for creativity based on their individual interests.

3 Our design approach

To illustrate our approach, we present and discuss the design and implementation of an interactive worked example that introduces students to the R programming language using data sourced from the NASA POWER (Prediction Of Worldwide Energy Resources) API. This interactive worked example was implemented within an introductory-level data science course at a large urban university during 2025.

At the start of the worked example, students are introduced to the NASA POWER website (power.larc.nasa.gov), and engage with a short video where they learn how to define a location using GPS co-ordinates and use GUI-driven tools on the website to generate a time series plot of climate data. After selecting a location of their own choice, students are then introduced to the first programming plan, which provides the code with a single "code blank" for sourcing and visualising

data from just one location. After learning about the use of quotes for strings when writing R code, students are provided with an adapted programming plan, which introduces a vector as a way to source and visualise data about two different locations (Figure 1).

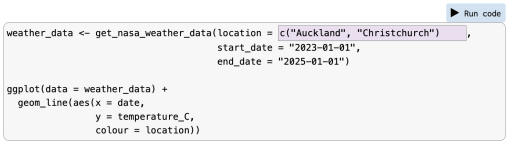

Figure 1: An example of a "code blank" interactive coding worked example, where students are introduced to a vector within the context of sourcing and visualizing temperature data from two different cities within New Zealand.

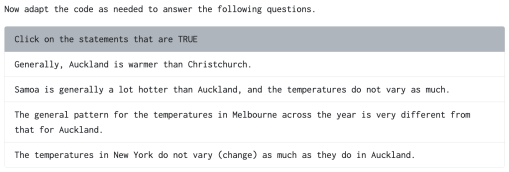

Figure 2: An example of a "tinker question", where students need to change the code to source and visualise temperature data from the two locations referred to in each statement, so that they can verify if the statement is TRUE or FALSE.

The interactive worked example then uses a "tinker question" to encourage students to change the code blank in order to simultaneously practice writing the correct syntax for vectors and learn more about the temperatures of different locations around the world.

The remainder of the interactive worked example is designed in a similar structure, where the initial programming plan is adapted or expanded based on modifying "code blanks". The worked example ends with students being given the complete programming plan with many code blanks, which they are asked to fill out in order to visualise data for two new locations, after first describing why they chose these locations and what they thought they might see when comparing them across a new climate-related variable not yet explored. In this way, the students construct their own complete worked example, based on their own personal interests, which can then be used to help them develop code for their project task.

4 Discussion

The overarching goal of our research is to frame the learning of computer programming for introductory-level data science students in an epistemic sense. Therefore, we aim at all students to participate, promote a tinkering mindset, encourage curiosity and creativity about the world, and value diverse perspectives. To reach this goal, we propose the following four design principles for interactive worked examples within our courses. The interactive worked examples:

1. should present a cohesive and carefully sequenced collection of programming plans, which can be combined to solve a particular problem.
2. should use "code blanks" that indicate parts of the code to be changed when using it for one's own goals.
3. should use "tinker questions" or instructions that guide students to complete or change the code blanks and then reflect on what is produced.
4. should be followed up by students working on their own project, which is similar in structure, but based on their individual interests.

We have found these design principles to be effective in our teaching practice, supported by positive learning reflections written by students who have completed the interactive worked examples. We plan to use students' eye-tracking, digital interaction, and interview data to investigate whether and how students found the interactive worked examples constructed from these design principles supportive in their individual programming learning processes.

Conclusion. In summary, we believe that the use of well-designed interactive worked examples within data science courses is essential for being introduced to (epistemic) programming, even in an era of Generative AI, as carefully structured tasks can provide conceptual and pedagogical support that is not easily replicated by automated tools alone.

References

Robert K. Atkinson, Sharon J. Derry, Alexander Renkl, and Donald Wortham. 2000. Learning from examples: Instructional principles from the worked examples research. *Review of Educational Research*, 70(2):181–214, June. 01408.

Kathryn Cunningham, Barbara J. Ericson, Rahul Agrawal Bejarano, and Mark Guzdial. 2021. Avoiding the turing tarpit: Learning conversational programming by starting from code's purpose. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, page 1–15, Yokohama Japan, May. ACM.

Anna Fergusson. 2023. Designing positive first experiences with coding for introductory-level data science students. In *Proceedings of the Satellite Conference of the International Association for Statistics Education (IASE) — IASE 2023: Fostering Learning of Statistics and Data Science*, Toronoto, Canada, March. Published 29 March 2024.

Sven Hüsing, Carsten Schulte, and Felix Winkelnkemper. 2023. Epistemic programming. In Sue Sentance, Erik Barendsen, Nicol R. Howard, and Carsten Schulte, editors, *Computer Science Education: Perspectives on Teaching and Learning in School*, pages 291–304. Bloomsbury Academic, London, 2 edition.

Alexander Renkl, Robert K. Atkinson, and Cornelia S. Große. 2004. How fading worked solution steps works – a cognitive load perspective. *Instructional Science*, 32(1/2):59–82, January.

John Sweller and Graham A Cooper. 1985. The use of worked examples as a substitute for problem solving in learning algebra. *Cognition and instruction*, 2(1):59–89.